Fan-Out on Write vs Read

push for most, pull for celebrities

Watch the 4-min video

In one line: fan-out on write pushes each new post into every follower's precomputed timeline (cheap reads, expensive writes); **fan-out on read** assembles the feed when it is opened (cheap writes, expensive reads). Real systems use a **hybrid**: push for ordinary accounts, pull for celebrities, merged at read time.

1 200 followers or 100 million

A regular user with 200 followers posts a photo. A pop star with 100 million followers posts one too. Treat both the same and one is trivial while the other means 100 million timeline writes within seconds. The question is **when** you pay for assembling feeds: when someone writes, or when someone reads.

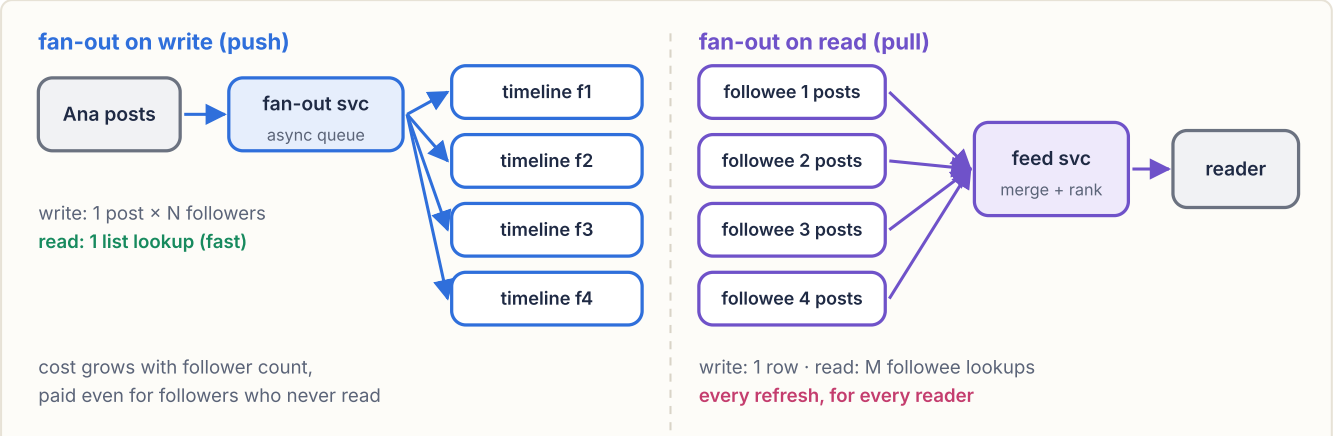


Figure 1. The two extremes. Push moves the work to the writer; pull moves it to every reader.

2 Fan-out on write (push)

When Ana posts, the post is stored once, and a background **fan-out service** reads her follower list and appends the post ID to each follower's timeline (for example a Redis list per user, trimmed to a few hundred entries). Reading a home feed is then a single list lookup plus hydration (lesson 29). Because reads vastly outnumber writes in social apps, this is usually the right default.

The numbers that made this famous come from Twitter (around 2012, as described in *Designing Data-Intensive Applications*, ch. 1): about **4.6k tweets/s** on average (12k+ at peak) against **300k home-timeline reads/s**. With an average of ~75 followers, push meant ~345k timeline inserts/s, which is far cheaper than doing 300k multi-way merges a second.

3 Fan-out on read (pull)

Posting is one write to the author's own post list. Opening the feed fetches recent posts from every account you follow, merges them by time and ranks them. Following 1,000 accounts means up to 1,000 lookups (batched or scatter-gathered) per refresh. Pull wastes nothing on inactive followers and new posts are visible instantly, but read latency and load grow with the number of followees and the refresh rate.

4 Where each one breaks

	Push (on write)	Pull (on read)
Read cost	O(1) list read	O(followees) lookups + merge
Write cost	O(followers) inserts	O(1)
Freshness	delayed by **fan-out lag**	immediate
Storage	timeline per user (IDs only)	none extra
Breaks for	celebrities: millions of writes per post, most to inactive users	heavy readers and users who follow thousands
Deletes / unfollows	must remove IDs or filter at read time	naturally correct

Push for a 100M-follower account means a burst of 100M inserts. Even at 1M inserts a second that is minutes of **fan-out lag**, during which some followers see the post and others don't, and the queue delays every other user's posts too. Pull for everybody means the busiest accounts' post lists become hot keys read millions of times a second.

5 The hybrid

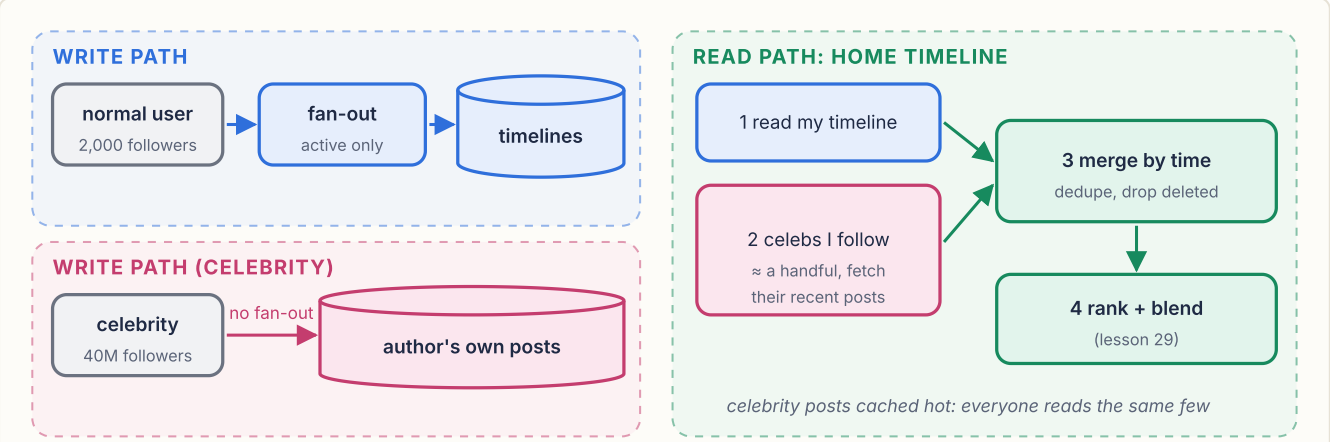


Figure 2. Ordinary authors are pushed; celebrity posts stay with the author and are merged in when a follower reads.

Pick a threshold on follower count (tunable; something like hundreds of thousands). Below it, posts fan out on write. Above it, posts are **not** fanned out. On read, the feed service takes your precomputed timeline, fetches recent posts from the handful of celebrities you follow (from a hot cache, since millions of readers ask for the same few lists), merges by time, and ranks. Most users follow few celebrities, so the read-time merge is small and bounded.

6 Making it efficient

- **Asynchronous:** the post API writes the post and enqueues one fan-out job, then returns. Workers split big follower lists into chunks (e.g. 5,000 followers per task) and process them in parallel.
- **Skip inactive users:** don't push to accounts that haven't opened the app in, say, 30 days; rebuild their timeline by pull when they return.
- **Short timelines:** keep only the latest few hundred IDs (Twitter kept about 800); older pages fall back to pull or a search index.
- **Cap per-post work** and prioritise: fan out to the most active followers first.

- **Handle the edges:** a new follow back-fills recent posts from that account; an unfollow or delete is filtered at read time, with lazy cleanup, instead of a second fan-out storm.
- **Measure fan-out lag:** the time from post to the last follower's timeline insert, as an SLO.

WORKED EXAMPLE: TWITTER'S APPROACH

A regular user with 2,000 followers tweets. The tweet is stored, and the fan-out service inserts its ID into the cached home timelines of the ~1,200 followers active this month: 1,200 small list pushes, done within a second or two. A celebrity with 40 million followers tweets: it is stored in their own tweet list and nothing is fanned out. When Ben loads his home timeline, the service reads his cached timeline (up to ~800 IDs), fetches the latest tweets of the 6 celebrities he follows from a hot cache, merges both lists by time, removes deleted or muted items, ranks, and hydrates the first page.

TRADE-OFFS

- A lower celebrity threshold reduces write storms but makes more reads merge more lists.
- Skipping inactive users saves most writes but makes a returning user's first load slow.
- Long timelines allow deep scrolling from cache but multiply memory across all users.
- Push gives predictable read latency; pull gives perfect freshness and simpler deletes.

COMMON MISTAKES

- Fanning out synchronously inside the post request.
- Storing full posts in timelines instead of IDs (edits and deletes become impossible).
- Treating the celebrity threshold as fixed: accounts cross it, so the read path must handle both kinds.
- Forgetting the hot-key problem on the pull side: celebrity post lists need replicated caching.

7 Key terms

Fan-out on write: push post IDs into followers' timelines when posted

Hybrid fan-out: push for normal users, pull for celebrities

Home timeline: precomputed list of post IDs for one user

Fan-out on read: build the feed from followees' posts when opened

Fan-out lag: time until a post reaches every follower's timeline

Celebrity threshold: follower count above which posts are pulled

8 Interview questions

Q1. Compare fan-out on write and fan-out on read. BASIC

Write pushes each post into every follower's timeline: fast reads, expensive writes. Read assembles the feed when it is opened: cheap writes, expensive reads.

Q2. How do you handle celebrities? BASIC

Hybrid: push posts from normal users, and pull celebrity posts at read time, merging them into the precomputed timeline.

Q3. Reads are 100× writes and the average user has 200 followers. Which model? SCENARIO

Push: 200 cheap inserts per post buys O(1) reads for a workload dominated by reads; add pull only for very large accounts.

Q4. A user returns after six months. What happens to their feed? SCENARIO

They were skipped by fan-out, so the timeline is rebuilt on demand by pulling recent posts from followees, then normal push resumes.

Q5. A post is deleted after being fanned out to 2 million timelines. How do you remove it? TRICKY

Don't fan out a delete storm: mark the post deleted, filter it at hydration or merge time, and let timelines trim it lazily.

Q6. Why does push give worse freshness than pull for big accounts? TRICKY

Fan-out takes time proportional to followers, so some followers get the post minutes after others; pull reads the author's list directly and is immediate.

Next: SD-Q-006 · Batch Processing